

6. Implementation Architecture & UDIF Interchange

6.1 Closure Algorithms and Complexity Analysis

We now formalize algorithmic procedures for each of the nine closure operators $\{\text{varsigma}_a, \text{varsigma}_g, \text{varsigma}_s, \text{varsigma}_{\ell}, \text{varsigma}_{\text{sym}}, \text{varsigma}_{\text{caus}}, \text{varsigma}_{\text{ctx}}, \text{varsigma}_{\text{coh}}, \text{varsigma}_{\text{traj}}\}$. For a given system object $X \in \mathcal{MMK}$, each closure varsigma_i produces a quotient system $\text{varsigma}_i(X)$ by identifying states of X that are equivalent under the corresponding relation (algebraic, geometric, spectral, etc.). We specify for each varsigma_i : the input assumptions on X (e.g. required structures like a metric or group action), a step-by-step algorithm for computing the closure, the formal output (the reduced quotient system $X_i = \text{varsigma}_i(X)$), and asymptotic complexity in terms of relevant parameters (number of states $N=|X|$, number of transitions E , group size $|G|$, trajectory length T , etc.). We also discuss practical bottlenecks and the distinction between exact and approximate implementations for each case.

Algebraic Closure (varsigma_a): *Behavioral Equivalence*.

Input: A metric-measure-kernel system $X = (|X|, d_X, \mu_X, K_X)$ with a **state-transition graph** (or Markov chain) defined by the kernel $K_X: |X| \times |X| \rightarrow [0,1]$. No additional structure is needed beyond the Markov transitions.

Output: The quotient system $\text{varsigma}_a(X)$ in which states that exhibit the same *behavioral patterns* are merged. Formally, $\text{varsigma}_a(X)$ identifies states $x_1 \sim_a x_2$ if for every possible sequence of transitions (of any finite length), the probability distributions of outcomes from x_1 and x_2 are identical ¹. Equivalently, $\text{varsigma}_a(X)$ yields **behavioral equivalence classes** of states. This closure can be viewed as producing the coarsest **bisimulation** partition of the Markov process.

Algorithm: We employ a **partition-refinement** procedure (akin to the Paige-Tarjan algorithm for bisimulation minimization ²). Initially, group states by their immediate transition probabilities (or by any observational distinctions given by μ_X). Then iteratively refine the partition: at each step, if two states in the same block have differing transition probabilities into some other block, split them into separate blocks. This procedure converges to the minimal stable partition under which all states in a block have indistinguishable transition behavior. Data structures like a union-find (disjoint-set forest) can efficiently maintain the merging and splitting operations ³.

Complexity: The refined bisimulation algorithm runs in *nearly linear* time in the size of the transition graph. For N states and E transition edges, an optimized implementation achieves $O(N \log N)$ time for sparse graphs ². If transitions are dense, complexity can approach $O(N^2)$. The space complexity is $O(N + E)$ to store the graph and partitions.

Bottlenecks & Approximations: In the worst case (very large N or high branching), exact behavioral equivalence computation can be expensive. A practical bottleneck is that distinguishing states may require considering long transition sequences. In large or continuous-state systems, one may resort to *approximate algebraic closure* by truncating the lookahead (only refining up to a fixed path length) or by clustering states with similar but not identical behavior. Such approximations sacrifice the guarantee of true bisimulation but can dramatically reduce computational load. For finite Markov processes, however, the exact partition refinement is usually feasible and yields a provably minimal quotient preserving all finite-horizon dynamics

¹.

Geometric Closure ($\text{\textbackslash varsigma_g}$): *Recurrence and Connectivity*.

Input: A system X with an underlying **directed graph structure** (states as vertices, transitions as edges). Typically, X is assumed to have a notion of reachability or a topology on $|X|$ induced by K_X or a distance d_X . In many cases, $\text{\textbackslash varsigma_g}$ uses just the directed graph of reachable transitions.

Output: The quotient system $\text{\textbackslash varsigma_g}(X)$ in which states are merged according to **recurrence connectivity**: $x_1 \sim_g x_2$ if and only if x_1 and x_2 belong to the same strongly connected region of the state-space graph ¹. Intuitively, any two states that can mutually reach each other (perhaps via intermediate states) are identified, since they lie in the same recurrent class. Transient states (those that cannot return once left) may form singleton classes or be separated. Formally, $\text{\textbackslash varsigma_g}(X)$ collapses each **strongly connected component (SCC)** of the transition graph into a single meta-state, preserving the transition structure between components.

Algorithm: We perform a graph traversal to find strongly connected components. A classic method is **Tarjan's algorithm** or **Kosaraju's algorithm**, which performs a depth-first search to identify SCCs in linear time ⁴. Concretely, one can assign each state a discovery index and low-link value via DFS and identify root nodes that indicate an SCC, or run a DFS from each node in the transpose graph. The result is a partition of $|X|$ into disjoint SCCs. The quotient $|X|/\sim_g$ is then constructed by treating each SCC as a single state; the distance $d_{\{g\}}$ on the quotient can be inherited or defined as the minimal distance between representatives (if a metric exists), and the measure μ_g and kernel K_g are obtained by aggregating transitions among components.

Complexity: Finding all SCCs is linear in the size of the graph: $O(N + E)$ time for N states and E transitions ⁵. This is efficient even for large systems, provided the graph is not extremely dense. Memory is $O(N+E)$.

Bottlenecks & Approximations: Exact geometric closure via SCC is straightforward and efficient for finite discrete systems. However, in systems with a continuous or metric space structure (e.g. a continuous state space where $\text{\textbackslash varsigma_g}$ might represent quotienting by connected regions in a metric topology), an exact computation may involve finding connected components in a continuous space, which can be non-trivial. In such cases, one may approximate $\text{\textbackslash varsigma_g}$ by **coarse clustering**: e.g. cluster states that are within a small distance ϵ of each other or that are connected via high-probability paths. For example, in a molecular configuration space one might cluster nearly connected states rather than strictly requiring mutual reachability. Another scenario is in Hamiltonian or symplectic systems, where an *exact geometric reduction* might require computing a symplectic quotient (preserving a geometric structure exactly), whereas a practical approximation might use simpler metric clustering or dimensionality reduction ⁶ ⁷. Thus, $\text{\textbackslash varsigma_g}$ can be implemented exactly for discrete graphs (via SCCs) or approximately for complex metric spaces by using algorithms like density-based clustering to find approximately connected regions.

Spectral Closure ($\text{\textbackslash varsigma_s}$): *Slow Dynamical Modes*.

Input: A system X with a well-defined linear operator representing its dynamics. Typically this is the **transition matrix** T of the Markov kernel K_X , or a Laplacian/operator on the state graph. We assume T (of size $N \times N$) can be constructed or accessed, and perhaps that X has a metric or inner product allowing spectral decomposition.

Output: The quotient system $\text{\textbackslash varsigma_s}(X)$ obtained by identifying states that are **spectrally similar** with respect to the dynamics. Concretely, $\text{\textbackslash varsigma_s}$ merges states that cannot be distinguished by the leading eigenmodes of the transition operator ¹. Intuitively, if the long-term evolution or “slow” dynamics of the Markov chain weights two states nearly identically (for instance, they belong to the same metastable cluster), those states are merged. The quotient often corresponds to clusters in state space that have

coherent behavior under the dynamics (e.g. metastable states in a Markov process).

Algorithm: We employ **spectral clustering** on the state-space graph or transition matrix ⁸. First, compute the top k eigenvalues and eigenvectors of the transition matrix T (or of a normalized graph Laplacian $L = I - T$) that capture the most significant slow dynamics (the choice of k may depend on a desired number of clusters or spectral gap). This can be done via algorithms like **Lanczos iteration** or randomized SVD for large sparse matrices. Next, treat the leading k eigenvectors as embedding coordinates for the states in $\mathbb{R}^k$. Finally, run a clustering algorithm (such as k-means or hierarchical clustering) on these k -dimensional points to group states. The resulting clusters define an equivalence relation $\sim_s$; we collapse each cluster to a single state in $\Sigma(X)$. The reduced transition probabilities K_s can be obtained by aggregating T over these clusters (and possibly re-orthonormalizing to ensure a valid Markov chain).

Complexity: Computing k leading eigenvectors for a sparse matrix with N states and E nonzero transitions typically costs about $O(k \cdot (N + E))$ using Lanczos or power iteration ⁸. In practice, this might be $O(kN)$ for sparse graphs if E scales with N . The clustering step (k-means) adds an additional $O(kN \cdot \text{iter})$, which is usually smaller order. Thus overall spectral closure can be done in roughly linear time for fixed k . Memory usage is $O(N + E + kN)$.

Bottlenecks & Approximations: The main cost is the eigen-decomposition of a potentially large matrix. For extremely large N , even $O(N^2)$ operations might be infeasible. Approximation is inherent in choosing a small number of modes k — if important dynamics lie in discarded eigenvectors, some distinction between states is lost. In practice, one selects k based on a spectral gap or desired granularity. This is an *approximate* closure unless k is large enough to capture all significant variation. However, it provides a continuous trade-off between reduction and accuracy. If needed, one can refine by increasing k . In summary, Σ_s is often implemented inexactly (truncated spectrum) for tractability, but if N is small one could in principle compute the full eigenbasis for an exact identification of *all* spectrally indistinguishable states (which in the extreme case would just be the identity if all eigenvectors are used).

Logical Closure (Σ_{ℓ}): Indistinguishability by Observables/Predicates.

Input: A system X equipped with a set of **observables or logical predicates** $\{\varphi_i \mid i \in I\}$ that one can evaluate on states. This could come from a logical language (e.g. propositional or temporal logic) associated with the system or simply from measurement functions on states. We assume each state $x \in |X|$ has a definable truth value or outcome for each predicate (for example, a vector of boolean or real observation values).

Output: The quotient $\Sigma_{\ell}(X)$ where states are merged if they are **logically indistinguishable** with respect to the given set of predicates ⁹. Formally, $x_1 \sim_{\ell} x_2$ if for every predicate φ in our specified set, $\varphi(x_1)$ and $\varphi(x_2)$ are the same. Equivalently, no formula or observation in the considered language can distinguish x_1 from x_2 . This is analogous to identifying states that satisfy exactly the same atomic propositions (or have identical observation outcomes), a concept used in model reduction (e.g. partitioning by observation equivalence or ω -bisimulation in logical systems). The output $\Sigma_{\ell}(X)$ has state space equal to the set of distinct truth assignments to all predicates, and inherits dynamics consistent with those assignments (transitions between logical equivalence classes occur if an example state from one class can transition to a state in another class under K_X).

Algorithm: We can compute logical equivalence by a straightforward **partitioning by predicate values**. Initially, partition $|X|$ into blocks such that within each block, all states share identical values for all primitive predicates φ_i . (This is effectively grouping by the vector of predicate outcomes.) If the logical language allows compound statements or temporal operators, a more involved fixed-point may be needed: one can iteratively refine the partition if future behavior combined with logic distinguishes states (similar to the behavioral partition but now guided by logical formulas). For static properties, one pass over

all states to evaluate $\{\varphi_i\}$ and group accordingly suffices. Alternatively, this problem can be phrased as a SAT/SMT solving task: ask whether there exists a formula that distinguishes two given states; if not, they remain in the same class ¹⁰. In practice, constructing the equivalence classes directly by evaluation is simpler for a finite predicate set. Advanced approaches might use a decision tree or binary decision diagram representing the predicates to cluster states. If the logic is temporal (involving sequences of states), an algorithm akin to partition refinement (like modal bisimulation algorithms) would be used, which alternates between checking one-step logical distinguishability and refining.

Complexity: If I (the number of predicates) is fixed or grows slowly with N , grouping states by predicate evaluations is $O(N \cdot |I|)$ (essentially linear in N) plus the cost of evaluating each predicate on each state. If predicates are simple properties of the state, this is efficient. The worst-case complexity arises if we treat the problem as solving many logical distinguishability queries, which can be NP-hard (since finding a formula that distinguishes two states can reduce to a SAT problem) ¹¹. Indeed, the general logical equivalence (especially with rich logics or if $|I|$ scales with N) is NP-complete in the size of the logical description. However, for *bounded* logics or a fixed set of observations, the partition can be computed in polynomial time. In summary, σ_{ℓ} is tractable for moderate predicate sets or simple logics, but in general one must restrict the logic to avoid exponential blowup.

Bottlenecks & Approximations: The primary challenge is if the logical language is very expressive (e.g. first-order logic with quantification over paths, etc.), in which case determining equivalence of states might require considering arbitrarily complex formulas. In practice, one limits the observation/predicate set to those of interest. An approximate logical closure might involve ignoring some predicates (merging states that differ on very complex or insignificant conditions) or bounding the logical depth (e.g. only distinguish states by predicates up to a certain formula size). This yields a coarser but computationally manageable equivalence. Another approximation approach is to use machine learning to group states by similarity in predicate outcomes (for example, treat the vector of predicate truth values as a feature vector and cluster them in a Hamming metric), which might merge some logically distinguishable states if the differences are minor or rare. When implemented exactly, σ_{ℓ} ensures that all observable properties are invariant in each quotient class, which is crucial for model reduction in contexts like formal verification.

Symmetry Closure (σ_{sym}): Quotient by Group Actions.

Input: A group action on the state space of X . Specifically, assume a group G (often a finite or compact Lie group) acts on $|X|$: for each $g \in G$ and state $x \in |X|$, there is a state $g \cdot x \in |X|$. We require that this action is an **automorphism** of the system structure, i.e. it preserves the metric, measure, and transition kernel: $d_X(g \cdot x_1, g \cdot x_2) = d_X(x_1, x_2)$, $\mu_X(g \cdot A) = \mu_X(A)$, and $K_X(g \cdot x, g \cdot A) = K_X(x, A)$ for all states and measurable sets A . Invariant structures like this arise from symmetries of the system (permutations of identical particles, gauge symmetries in physics, etc.).

Output: The quotient $\sigma_{\text{sym}}(X) = X/G$, the **orbit space** of states under the group action ¹² ¹³. States x_1, x_2 are equivalent ($x_1 \sim_{\text{sym}} x_2$) iff there exists some $g \in G$ such that $x_2 = g \cdot x_1$ ¹⁴. In other words, they lie in the same orbit of G . The reduced system has state space $|X|/G$ (each state is an orbit $[x]$), a metric d_{sym} induced by taking the infimum distance over representatives of orbits ¹⁵, a measure μ_{sym} which is the pushforward of μ_X under the quotient map $x \mapsto [x]$, and a kernel K_{sym} defined by integrating transitions over group elements (if x transitions to set A with probability p , then $[x]$ transitions to $[A]$ with the appropriately averaged probability) ¹³. Intuitively, σ_{sym} “forgets” redundant distinctions between states that are related by a symmetry of the system.

Algorithm: To compute X/G , we enumerate the orbits of G acting on $|X|$. A basic procedure is: initialize all states as unvisited; then for each unvisited state x , perform a group orbit traversal (e.g.

depth-first search or BFS over the graph of group actions): compute $\{g \cdot x : g \in G\}$ which is the orbit of x , mark all those states as visited and label them as one equivalence class. Then continue to the next unvisited state. This yields a complete partition of $|X|$ into orbit classes. In implementation, if G is finite and given by generators, one can generate the orbit by iteratively applying generators. For efficiency, one often uses union-find: initially each state is its own set, then for each generator g and state x , union the sets containing x and $g \cdot x$. This effectively merges all orbits without redundant traversal. In cases where G is large but structured (like a Lie group), one may need to represent states symbolically to identify orbits without brute force enumeration. There are software tools (e.g. **GAP** or **SageMath** for permutation groups) that can assist in orbit finding ¹⁶.

Complexity: If G is finite, the worst-case time is $O(N \cdot |G|)$, as in the worst case one might apply each group element to each state. However, often $|G|$ is huge (possibly exponential in some encoding), so practical algorithms exploit group structure. For instance, if G is given by a set of generators much smaller than $|G|$, orbit search via generators can be considerably cheaper than iterating over all group elements. In many scenarios, orbits cover large portions of $|X|$ so the number of orbit representatives is $N/|G_{\text{avg}}|$ where $|G_{\text{avg}}|$ is average orbit size. In such cases, complexity is roughly $O(N + \text{orbit traversal steps})$. The complexity can thus be polynomial in N for fixed or moderately growing $|G|$. However, in the worst theoretical case (if G is extremely large relative to N or if applying a group element is expensive), the algorithm could be exponential in $\log |G|$. Memory is $O(N)$ for state markings plus storing a dictionary of representative mappings.

Bottlenecks & Approximations: A key bottleneck is handling very large or continuous groups. If G is continuous (like $SO(3)$, etc.), one cannot enumerate all group elements; one must rely on analytical insight to describe orbits (e.g. orbits might correspond to invariants like energy or momentum values). For large finite groups (e.g. permutation groups of many particles), specialized group algorithms (stabilizer chains, Schreier-Sims algorithm) can find orbits more efficiently than naive traversal. Approximate symmetry closure is less common, since symmetry is usually exact by definition. However, in cases where the symmetry group is not explicitly known or the system is "nearly symmetric", one might identify approximate orbits: for example, cluster states that are close under some transformations even if not exactly related by a known group element. This could be done by heuristics (like treating large but finite tolerance as equality). Generally, though, $\text{varsigma}_{\text{sym}}$ is applied in contexts where exact symmetry is known *a priori*. If the group action is too expensive to apply exhaustively, one might apply a partial closure: e.g. only quotient by a subgroup of G or only some symmetric properties, to reduce complexity.

Causal Closure ($\text{varsigma}_{\text{caus}}$): Reduction to Causal Structure Equivalence.

Input: A system X that includes a **causal model** or at least data from which causal relationships can be inferred. This often means X is a stochastic dynamical system where one can observe histories and interventions. Formally, one might assume for each state x we have information about its **ancestral history** (states that can lead to x) and its **future influence** (distributions of outcomes if starting from x under various interventions) ¹⁷ ¹⁸. In practice, input could be a dataset of trajectories and interventions, or a fully specified Markov decision process.

Output: The quotient $\text{varsigma}_{\text{caus}}(X)$ in which states are merged if they have the **same causal roles** in the system. A rigorous criterion is: $x_1 \sim_{\text{caus}} x_2$ if for every possible *intervention* or experiment on the system, the probability distributions of outcomes from x_1 and x_2 are identical ¹⁸. Equivalently, x_1 and x_2 have identical structural behavior in terms of cause and effect – they occupy the same position in the causal graph of the system. In practice, this means they share the same *Markov blanket* or conditional independence relationships with all other variables. The reduced system $\text{varsigma}_{\text{caus}}(X)$ abstracts away differences that do not affect how the system responds to or propagates causes. It can often be represented as a **causal DAG** (directed acyclic graph) or a simplified

influence network capturing the essential causal structure ¹⁹ .

Algorithm: Determining causal equivalence is challenging and often done via **causal discovery algorithms** on data ²⁰ . A common approach is the **PC algorithm** (Peter-Clarke) or variants like FCI (Fast Causal Inference), which analyze conditional independencies in observational (and optionally interventional) data to construct a causal graph. To apply this as a closure: 1. Use the data or model of $\mathcal{X}$ to infer the minimal causal graph (nodes = states or state variables, edges = direct causal influence). 2. In that graph, identify nodes (states) that are indistinguishable in terms of their parent and child sets in the graph and the statistical relationships – in other words, collapse nodes that share the same ancestors and descendants structure. This can involve merging nodes that form *equivalence classes* under the DAG (for example, in a Bayesian network two variables that always take on the same distribution given parents might be merged, or two states that act as identical mediators). 3. The result is a quotient graph (a minimal DAG) where each node is a class of original states. This quotient is $\Sigma_{\text{caus}}(\mathcal{X})$.

In simpler terms for a Markov chain: one could merge states that have **identical future transition distributions and identical reachable past**. Indeed, the theoretical definition given earlier requires identical arrival time distributions and future response kernels ²¹ ²² . To check this directly: for each pair of states, compare their conditional transition probabilities under all interventions; if equal for all tests, they are equivalent. This direct check is exponential in potential interventions, hence using structure discovery (PC/FCI) is more efficient by focusing on conditional independence patterns.

Complexity: Causal discovery algorithms like PC run in polynomial time under certain assumptions (e.g. bounded graph degree) but in the worst case can be super-polynomial or exponential in N (since the space of possible causal graphs is huge) ²³ . If we consider a fully known model, checking equivalence for each pair of states across all interventions is also combinatorially explosive in general. However, if the causal graph of the system has $m \ll N$ latent “macro-variables” (like factors or communities), Σ_{caus} will compress N drastically, and the complexity is dominated by inferring those m factors. In data-driven scenarios, PC algorithm complexity is roughly $\mathcal{O}(N^q)$ for some small q (related to the max number of parents), which is feasible for moderate N . The output size is the number of causal classes which is typically much smaller than N .

Bottlenecks & Approximations: Exact causal equivalence requires infinite data or a perfect model. In practice, one uses statistical tests to infer causal links with some error tolerance. This means Σ_{caus} is often *probabilistic*: we identify equivalences with high confidence rather than certainty. Approximations include restricting to a simpler model class (e.g. assuming linear relationships or no unobserved confounders to simplify discovery), or focusing on a subset of variables. One may also merge states that are *approximately* causally identical – for instance, if two states have very similar influence on outcomes in all tested scenarios, treat them as one (especially if the differences are within noise). Another practical strategy is to perform **dimensionality reduction** on the state transition history (using techniques like clustering on time-series response or autoencoders) to group states by how they react to inputs, which indirectly approximates Σ_{caus} . Thus, while Σ_{caus} has a firm theoretical definition, its implementation tends to rely on heuristic or statistical methods due to complexity. The end result is a simplified causal model where each equivalence class can be interpreted as a meta-state with a distinct role in the cause-effect structure of the original system ¹⁹ .

Contextual Closure (Σ_{ctx}): Context-Independence Quotient.

Input: A **context-dependent system** $\mathcal{X}_{\theta}$ with an external parameter (context) $\theta \in \Theta$ that modulates its dynamics ²⁴ . For example, a family of transition kernels $\{K_{X^{\theta}}^{\theta} : \theta \in \Theta\}$ is given, describing how the system evolves under different environmental conditions or parameters.

Additionally, a distribution ξ over contexts may be given (or one considers all contexts equally). In essence, each state x has a context-dependent behavior $K_X^{\theta}(x, \cdot)$, and $\text{varsigma}_{\text{ctx}}$ will identify states that behave the same *for all contexts*. We assume Θ is either finite or nicely parameterized so we can iterate or sample over it, and that we can evaluate or sample $K_X^{\theta}(x, \cdot)$.

Output: The quotient $\text{varsigma}_{\text{ctx}}(X_{\text{Xi}})$ where states are merged if they are **contextually equivalent**, meaning no context can differentiate them ²⁵. Formally, $x_1 \sim_{\text{ctx}} x_2$ if for every context $\theta \in \Theta$, the transition probabilities (or outcome distributions) from x_1 and x_2 are identical: $K_X^{\theta}(x_1, \cdot) = K_X^{\theta}(x_2, \cdot)$ for all θ (or at least, their expectation over any context distribution is identical ²⁵). Thus, x_1 and x_2 have the same *context-invariant behavior*. The reduced system has states representing these context-invariant classes. Such a reduction yields a model that is *robust* to changes in context: the dynamics in the quotient do not depend on context (they represent behaviors that persisted across contexts) ²⁶.

Algorithm: The computation resembles a refinement procedure across contexts. We can proceed as follows: Start by assuming all states are in one big equivalence class. For each context $\theta \in \Theta$, consider the kernel K^{θ} . Partition states by their K^{θ} -transition behavior (two states in the same class must have exactly the same K^{θ} row). In practice, one can hash or compare the distributions $K^{\theta}(x, \cdot)$ for all x . If x_1 and x_2 differ for context θ , they must go to different classes. Do this for every distinct context value. The intersection of all these partitions (one for each θ) will yield the final equivalence: two states end up in the same class iff they were never distinguished by any context. If Θ is continuous or large, we cannot exhaustively iterate; instead, one might rely on a symbolic representation of context effects or sample a dense set of contexts to distinguish states. Data structures useful here include a table of state behaviors indexed by context. We can also frame this as a refinement algorithm: iteratively refine the partition by checking if any context induces a different outcome distribution for states in the same block, splitting as needed ⁶. Because contexts might induce only slight differences, we may incorporate a tolerance (for approximate closure) as described below.

Complexity: If Θ has P distinct values to consider (or we sample P contexts), and comparing two states' distributions is $O(N)$ (or using some metric), a naive approach is $O(P \cdot N^2)$. However, more efficiently, we can go context by context: for each θ , group states by their K^{θ} distribution. If comparing distributions can be done via hashing or sorted lists, one context pass is $O(N \log N)$ or better. For well-structured contexts (e.g. a limited number of qualitatively different regimes), the overall complexity can approach linear in $N \times P$ ⁶. The statement "linear in parameters for well-structured contexts" ⁶ indicates that if each context only induces a simple partition and contexts are not too numerous, the process scales benignly. In the worst case of infinitely many contexts or highly sensitive distributions, it could be infeasible to fully check. Memory use is $O(N \times P)$ in worst-case if storing each state's context-indexed behavior explicitly, but often we compress that.

Bottlenecks & Approximations: The main difficulty is handling a continuous or very large context space. If Θ is continuous (like a range of temperatures or time-varying conditions), one cannot explicitly iterate every context. In such cases, an *exact* $\text{varsigma}_{\text{ctx}}$ might require proving symbolically that two states have identical functional dependence on θ . An approximate approach is to choose a representative finite set of context scenarios $\{\theta_1, \dots, \theta_P\}$ and require equivalence only on those. This yields a coarser approximation: states might be merged because they behave similarly on tested contexts, even if a rare context could distinguish them. Another approximation is clustering the *context-response profiles* of states: treat the mapping $\theta \mapsto K^{\theta}(x, \cdot)$ as a high-dimensional feature of x , and cluster these mappings (possibly by dimension reduction in context space). This could use techniques from robust control: e.g., measure the worst-case difference between two states' behavior over Θ , and merge states whose worst-case difference is below a threshold. This sacrifices absolute guarantees but ensures the reduced model is *approximately* context-invariant. If contexts are probabilistic

with distribution ξ , one can demand equivalence of the *average* behavior $\mathbb{E}_{\theta[K_X^{\theta}(x, \cdot)]}$ ²⁵, which is easier to compute; indeed the definition in the theoretical framework uses expected behavior over contexts. Implementations may thus focus on preserving *expected* behavior under context, which is a weaker condition that can be checked with fewer parameters (just the mixture kernel K_X^{ξ}). The fully robust version (all contexts identical) is stricter.

Cohomological Closure ($\text{varsigma}_{\text{coh}}$): *Topological Invariant Reduction*.

Input: A system X that has an inherent **topological or geometric structure** on its state space. In practice, we assume we can interpret $|X|$ as (or embedded in) a topological space, often via the metric d_X (e.g. treating states as points in a metric space or graph). We also assume we can construct a **simplicial complex or filtered complex** representing the shape of X (for instance, a Vietoris–Rips complex from the metric, or a state-graph viewed as a 1-dimensional complex). This lets us compute homology or cohomology groups $H_k(|X|)$, either directly or via persistent homology if the space is discrete with a notion of scale ²⁷. Essentially, X should be enriched with enough structure to talk about loops, holes, and higher-dimensional features in its state space or state-transition graph.

Output: The quotient $\text{varsigma}_{\text{coh}}(X)$ where states are merged according to **homological equivalence** classes ²⁷. One formal definition: compute the homology groups (or persistent homology) of the state space; two states x_1, x_2 are equivalent if they lie in the same support of homology classes ²⁸. In other words, they cannot be distinguished by any topological invariant or homology cycle. Equivalently, x_1 and x_2 share the same “homological signature”: for each homology class (cycle) in $H_*(|X|)$, *either both states participate in that cycle or neither does* ²⁹. For example, if the state space has a loop (1-dimensional hole) that passes through certain states, all states along that loop might form an equivalence class since they contribute to the same topological feature. The reduced system can be thought of as consisting of homology-support classes (sometimes called “homological sectors”), with dynamics aggregated in a way that preserves the presence or absence of each topological feature ³⁰. This closure ensures that topologically fundamental distinctions (like being in different connected components or different holes) are retained, but incidental spatial distinctions are removed.

Algorithm: We leverage persistent homology computation to extract topological features, and then map states to those features. A practical pipeline is: 1. Construct a filtered simplicial complex from the metric space of states (e.g. increase a radius parameter ϵ for Vietoris–Rips complex on points, or take graph distances in the state-transition graph). 2. Run a persistent homology algorithm (via libraries like Ripser or GUDHI) to compute the barcode of homological features across scales ³¹. This yields a set of significant homology classes (each perhaps represented by a cycle or collection of states). 3. For each homology class h identified (especially persistent ones that represent robust features), collect the set of states that participate in h (e.g. states lying on a cycle representing that homology generator). Each state can thus be characterized by a vector of booleans: which homology classes does it belong to? (For H_0 connected components, this is trivial: states in the same connected component share that class. For higher H_k , one needs representative cycles to assign membership – typically one can find an approximate support for each homology generator). 4. Define $x_1 \sim_{\text{coh}} x_2$ iff this vector is identical for x_1 and x_2 . Then form equivalence classes accordingly ²⁸. For example, all states in the same connected component and on the same loop might be one class, whereas states in a different connected component or not on that loop would be another. 5. The quotient system $\text{varsigma}_{\text{coh}}(X)$ has one state per unique homology-support pattern. We attach reduced metric/measures by inheriting or averaging over those classes. Notably, $\text{varsigma}_{\text{coh}}(X)$ preserves the persistent homology barcode of X by construction ³². This approach effectively partitions by topological features present in the state set. For discrete state graphs, a simpler special case is merging states by connected component (H_0 features) which coincides with part of varsigma_g ; $\text{varsigma}_{\text{coh}}$ generalizes this to higher-dimensional loops or voids.

Complexity: Computing persistent homology is the most expensive step. For N points, the worst-case complexity

of standard algorithms is $O(N^3)$ or more, but in practice modern tools handle thousands of points efficiently (often around $O(N^2)$ for moderate N)³¹. The complexity cited is roughly $O(N^2)$ for practical sizes³¹, which assumes the complex does not explode combinatorially. Memory is also a concern, as persistent homology may require storing complexes of size $O(N^2)$ (for Rips on dense data). After computing homology, mapping states to feature supports is at most $O(N \cdot f)$ where f is number of features (usually much smaller than N). If X is extremely large (millions of states), one might restrict to lower-dimensional homology or use approximations as below.

Bottlenecks & Approximations: The major bottleneck is the potentially high computational cost of homology. If the state space is high-dimensional or the distance calculations are expensive, one might approximate by restricting to lower homology dimensions (e.g. compute only connected components and loops, ignoring higher holes) or by subsampling states. An approximate cohomological closure* could cluster states by coarse topological features, for example: use a lower-resolution point cloud (coarse-grain the metric space by sampling or merging nearby points) to compute homology, then lift that partition back to the full set. Another simplification is to focus on certain known topological invariants (like the number of connected components or presence of one prominent loop) rather than the full barcode. This yields a coarser equivalence (merging states that might differ in some subtle topological feature if we chose to ignore it). On the other hand, if exactness is required, one must use efficient persistent homology libraries and possibly parallelize the computation. Fortunately, $\text{H}\text{om}_{\text{coh}}$ is often applied to systems where N is not astronomically large or where the state space has some low-dimensional geometric embedding (common in topological data analysis contexts). By preserving topological holes and voids, this closure is particularly valuable in data science and computational biology contexts where the “shape” of data is important³³.

Trajectory Closure ($\text{H}\text{om}_{\text{traj}}$): Path Signature Equivalence.

Input: Rather than X alone, we consider its **path space** $\Pi(X)$ – the set of possible trajectories (histories) that the system can take³⁴. In formal terms, $\Pi(X)$ might be the set of infinite sequences $(x_0, x_1, x_2, \dots)$ with $x_i \in |X|$ and $K_X(x_i, x_{i+1}) > 0$ (allowable transitions)³⁴. Alternatively, one can consider finite trajectories of length T for some large T if needed. We assume we can sample or enumerate trajectories, or at least have a way to represent paths (possibly implicitly via the transition structure). The path space can be given a topology (e.g. product topology or a rough path metric) enabling us to compare entire trajectories. In addition, for continuous-time or continuous-space systems, one might consider geometric properties of paths (like integrals along the path).

Output: The quotient $\text{H}\text{om}_{\text{traj}}(X)$ where **entire trajectories** are the objects being identified. Two trajectories γ_1 and γ_2 are equivalent ($\gamma_1 \sim_{\text{traj}} \gamma_2$) if they have the same “shape” or **signature** in a precise sense³⁵. One definition given is that they are *signature-equivalent*, meaning all iterated integrals of their coordinate functions (or an analogous path signature as in rough path theory) are the same³⁵. Intuitively, this means the two paths trace out the same pattern in state space, possibly at different speeds. Another criterion mentioned is *rough-path equivalence*, which is a technical condition from rough path theory indicating the paths cannot be distinguished up to a certain Hölder roughness tolerance³⁵. In simpler terms for discrete systems, we might say γ_1 and γ_2 are equivalent if as sequences of states they are identical after some *permutation of transient details* – for example, if the sequences visit the same sets of states in the same order, or produce the same sequence of observations. The trajectory closure thus compresses the path space by merging trajectories that represent the same qualitative behavior. The reduced object $\text{H}\text{om}_{\text{traj}}(X)$ could be viewed as a *quotient automaton* or *abstracted protocol*: each equivalence class of paths might be represented as a single path pattern in the quotient, with transitions representing progression from one pattern segment to another. This closure is particularly useful in contexts like process mining, protocol verification, or reinforcement learning, where many different raw trajectories correspond to the same underlying behavior

or strategy ³⁶ .

Algorithm: Exhaustive comparison of infinite trajectories is impossible, so in practice one works with finite approximations: either truncate trajectories to a maximum length T , or impose an equivalence that can be checked incrementally. A concrete approach uses **path signatures** from rough path theory: for a given finite trajectory $\gamma: x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$, one computes its **signature**, which is a collection of iterated integrals or sums $\{\int_0^T X_{i_1} dX_{i_2} \dots dX_{i_k}\}$ capturing the shape of the path up to level k . There are algorithms in the time-series analysis community to compute these signatures up to a certain level k in $O(k \cdot T)$ time per trajectory. Our algorithm can then proceed by computing signatures for a representative set of trajectories and then clustering or identifying those with identical (or nearly identical) signatures ³⁵ . If the signatures are identical up to a chosen level, we declare the trajectories equivalent. Another method (if working with the sequence of states directly) is to define features like n -gram counts or patterns in the sequence and use those for equivalence. In formal language terms, one could construct a trie or automaton of all trajectories and then merge states in that automaton that lead to the same future behavior (this becomes analogous to algebraic closure but on the path-level automaton). In any case, because $\Pi(X)$ is huge, we usually *sample* trajectories rather than enumerate all. For example, generate M trajectories of length T under typical conditions or policies; compute their signatures or other invariants; then partition those M trajectories accordingly. This yields an empirical trajectory closure. The theoretical closure is the limit as $M, T \rightarrow \infty$ and signature level $k \rightarrow \infty$. Practically, even a moderate T and M can capture the dominant trajectory patterns.

Complexity: Let M be the number of trajectories sampled and T their length (assuming equal length for complexity). Computing a signature of length T to level k is $O(kT)$. Comparing or clustering M signatures (each a vector of length $\sim k$ or some polynomial in k) typically costs $O(M^2 \cdot k)$ for naive comparison or $O(M \log M)$ for clustering methods. Thus the procedure is roughly $O(M \cdot kT + M^2)$ in worst case. If we treat k and T as moderate constants, it's essentially $O(M^2 + MT)$. In many applications, M and T might be a few hundred or thousand, making this tractable. We can also reduce complexity by incremental merging: e.g., build a prefix tree of trajectories (cost $O(MT)$) and then merge equivalent subtrees – this is akin to constructing a minimal deterministic automaton and has complexity polynomial in the size of that automaton (which is $\leq M \cdot T$). Overall, trajectory closure can be done in *polynomial time for a given finite sample* of trajectories. The phrase “polynomial for moderate-length paths” ⁷ indicates that if T is not too large, the computations (which might be polynomial in T and possibly exponential in some small dimension of path features) are manageable. Memory usage is $O(MT)$ to store trajectories plus overhead for signature storage.

Bottlenecks & Approximations: By definition, we are almost always doing an approximation – since considering all infinite trajectories is not feasible, we approximate Σ_{traj} by limiting trajectory length or using partial invariants. The key approximation parameters are: the *length* of trajectories considered, the *feature richness* of the path signature (how high an order of integrals or what pattern length is considered), and the *sample size* of trajectories. If these are too low, distinct trajectory classes might erroneously collapse. If taken to extremes (very large T , high-order signatures, exhaustive coverage of $\Pi(X)$), the computation becomes infeasible. So one balances these. In reinforcement learning, for instance, one might cluster episodes by their outcome and significant events, effectively computing a trajectory closure that merges trajectories yielding the same cumulative reward and visiting the same critical states. Such a clustering might ignore small differences in sequence. Another possible approximation is using **edit distance** or a similarity metric between sequences, merging trajectories that are within a small edit distance of each other (which is a weaker condition than having identical signatures). This can be computed with algorithms like hierarchical clustering on sequence space. While not capturing all theoretical invariants, it often suffices to identify essentially identical behaviors. It's also worth noting that Σ_{traj} , unlike the other closures, yields a quotient that may not live in $\mathcal{MMK}$ (since its “states” are trajectory-classes, not original states). In advanced treatments, one considers an

extended category $\mathcal{MMK}^\pi$ of path-based systems³⁷, and indeed σ is a closure on that extended category³⁷. For implementation, we treat it as a separate module that post-processes trajectories. In summary, trajectory closure is powerful for compressing sequential information (e.g. compressing a long protocol to a few prototypical paths³⁸), but it is typically implemented in an approximate, data-driven manner due to the immense size of path space.

Summary of Complexity and Trade-offs: Each closure above provides a specific abstraction with its own computational profile. In practice, applying multiple closures in sequence can drastically reduce system size at each step. If $|\sigma_i(X)| = \lambda_i |X|$ for some reduction ratio $\lambda_i < 1$, composing closures multiplies these ratios. Indeed, one can often achieve exponential state compression by applying all nine closures in succession¹³⁹. However, the most computationally heavy closures (notably σ_{coh} and σ_{traj} , and sometimes σ_{ell} or σ_{caus} for complex logics) may need to be approximated to be practical. The above algorithms outline *exact* procedures under ideal assumptions; in real implementations a mix of heuristics (like clustering, sampling, truncation) are used to get *approximately idempotent* closures that still preserve the key invariants. Importantly, each σ_i is idempotent by construction (applying it twice yields the same result) and when the closures commute, the order of application does not matter for the final combined reduction⁴⁰⁴¹. This theoretical property allows an implementation to flexibly choose an order (often we apply them sequentially as listed, but any order yields the same $\mathcal{Q}_{\text{ext}}(X)$ up to isomorphism). Complexity analysis of sequential composition suggests that as the state space shrinks, later closures operate on a much smaller input, mitigating their high nominal cost. For example, one might apply σ_a and σ_g first to cut down N , then σ_s on a reduced matrix, then the heavier topological or trajectory analysis on a drastically smaller state set. This staged reduction pipeline is a key design principle of our implementation, as we describe next.

6.2 KUSMAN-Reduce Software Architecture

To implement the above closures and their compositions in a flexible way, we propose a modular software architecture called **KUSMAN-Reduce**. This library is designed with a **declarative, modular reduction philosophy**: each closure is a self-contained module with a uniform interface, and complex reductions are specified by composing these modules. The architecture emphasizes extensibility (new closures can be added), correctness (ensuring the algebraic laws of idempotence and commutativity are respected), and interoperability with existing scientific software. Below we outline the core components and design of KUSMAN-Reduce:

- **Closure Modules:** Each closure operator σ_i is implemented in its own module or class, providing a standard method (e.g. `apply()` or `reduce()`) that takes a system X and returns the reduced system $\sigma_i(X)$. All closure modules conform to a common interface, so they can be invoked and composed uniformly⁴². Internally, a closure module contains the algorithm described in Section 6.1 (possibly calling optimized libraries). For example, `AlgebraicClosure.apply(X)` will perform the partition refinement on X and return the quotient system; `GeometricClosure.apply(X)` runs the SCC algorithm, etc. These modules may be implemented in Python but delegate heavy computation to optimized backends (e.g. C++ for graph algorithms or Rust for heavy numeric tasks). The design encapsulates each algorithm's details, allowing them to be improved or swapped out independently.

- **Intermediate Representation (IR) Engine:** At the heart of the library is a data structure for **labeled MMK objects**, which we can call a **System IR**. This might be a class `System` or `MMKObject` that holds the components $(|X|, d_X, \mu_X, K_X)$ along with any additional annotations (like state labels or group actions). The IR provides methods to manipulate the system: merging states, relabeling, computing invariants, etc. For example, if a closure module decides to merge a set of states, it will call an IR method to actually perform that merger and update the internal representation. The IR is responsible for consistently updating the metric, measure, and kernel after operations. It also tags states with identifiers so that even after reduction, we maintain a mapping back to original states (facilitating provenance and quotient-map tracking). Essentially, the IR acts as an **intermediate language** in which all reductions are expressed – it is like an object model of MMK that can be manipulated by closures. Because we need to label equivalence classes, the IR might include a union-find structure or mapping from original state IDs to current representative IDs. This engine thereby enables representing not just a single system but potentially a lattice of systems: one can keep multiple reduced versions of X in memory, each identified by which closures have been applied. In implementation, `system_struct.py` serves this role, encoding metric-measure-kernel systems and supporting plugins for additional structure like group actions or context spaces ⁴³.

- **Reduction Pipeline Executor:** This component is responsible for orchestrating the application of multiple closures in sequence, according to a **declarative specification**. The user can specify a pipeline either programmatically (e.g. by passing a list of closure modules to compose) or via a configuration (a JSON or script listing which closures to apply). The pipeline executor (`compose()` in design) will take an input system and successively apply each closure in order ⁴⁴. It uses the uniform interface of closure modules, and in between, it may perform checks or record metadata. Crucially, the executor also handles **commutativity and idempotence checks**: after applying a closure, it can verify (in debug or verbose mode) that re-applying it yields no further change (ensuring idempotence), and it can optionally check if any previously applied closure would have given a different result if done now (to detect non-commutativity issues). Because the closures are theoretically commuting, such checks are mostly for catching implementation bugs or cases where we used approximations. The executor may optimize the pipeline by recognizing when one closure's effect subsumes another's (for example, if two closures commute, their order can be swapped or if one yields no change perhaps skip the other). This logic effectively **“hydrates the operator lattice”** ⁴⁵ – it is aware of the lattice of closure combinations and ensures the correct relationships. In future, this could allow exploring different sequences automatically. The pipeline executor ultimately outputs the fully reduced system (e.g. $\text{MMK}_{Q_{\text{ext}}}(X)$ if all closures are used, or a partial $\text{MMK}_{Q_S}(X)$ for some subset S of closures).

- **Commutation & Lattice Logic:** Although each closure is individually idempotent, verifying that different closures commute on a given dataset can be complex. KUSMAN-Reduce includes a **commutation checker** that can test a pair (or set) of closures on a sample system to ensure $\mu_i(\mu_j(X))$ matches $\mu_j(\mu_i(X))$ up to isomorphism. This can be done by comparing the outputs structurally (via a canonical form or hash of the reduced systems). A dedicated module (e.g. `validate.py`) provides functions to test **idempotence** and **pairwise commutativity** for any registered closures ⁴⁶. It can also verify the inclusion property (that the original X maps injectively into each closed version). These checks might be used in a testing suite or even at runtime if a user toggles a “verify” flag after a reduction pipeline. They ensure that any new closure added to the system does not violate the lattice properties. The architecture may also

maintain a representation of the closure lattice: for $n=9$ closures, there are 2^9 possible combinations (paths in the lattice). We can represent this lattice as a directed acyclic graph (DAG) where nodes are partial reductions and edges correspond to applying a closure. The **lattice-traversal logic** allows navigation in this DAG – for instance, the user might compute $\sigma_a(X)$ and $\sigma_g(X)$ separately and then meet (intersect) them by composing, or start from the full $\mathcal{Q}_{\text{ext}}(X)$ and inquire how to get to a less reduced version (this would correspond to going “up” the lattice by including fewer closures). The software can provide methods to **interpolate between closure sets**: e.g. given a reduced system with closures S applied and another with closures T applied, find the common refinement (which would be closures $S \cap T$ applied, or apply the missing ones as needed). Though applying closures in different orders yields isomorphic results, this lattice traversal feature is useful for exploring the effect of different combinations of closures on a model. It essentially gives the user the ability to turn certain reduction “on or off” and see the outcome, leveraging the order-independence proven in theory ⁴¹ ⁴⁷ .

- **Data Serialization Layer:** KUSMAN-Reduce is built to handle large systems, so efficient I/O is important. The architecture includes support for storing and loading systems (both original and reduced) in standard formats. Sparse matrices for K_X can be stored in CSR/COO format (using SciPy or similar), and large state labels or partitions can be stored in databases or binary files. We use formats like **HDF5** for array data and possibly graph formats (GraphML, adjacency lists) for the connectivity. A custom JSON schema is provided for small models, but for large ones a binary format or database is preferred ⁴⁸ . This serialization ties in with the UDIF format described in Section 6.3 for full interoperability. In implementation, we might have `system.save("file.udif")` and `System.load("file.udif")` methods.

- **External Library Integration:** Rather than reinvent algorithms, KUSMAN-Reduce leverages existing scientific libraries:

- Graph algorithms (for σ_a , σ_g): we integrate with **NetworkX** or graph-tool for operations like finding SCCs or performing BFS for symmetry orbits. For example, computing SCCs can call NetworkX’s `strongly_connected_components` function.
- Linear algebra and spectral: use **SciPy** (ARPACK) for eigen computations for σ_s , or potentially **networkx.algorithms.community** for spectral clustering.
- Topological data analysis: integrate with **scikit-tda** (which wraps GUDHI, Ripser, etc.) for persistent homology in σ_{coh} . We feed in distance matrices or point clouds from our state metric.
- Logical predicates and SAT: integrate with libraries like **SymPy** (for symbolic logic simplification) or **PySAT/Z3** for checking logical distinguishability in σ_{ell} . For instance, SymPy can represent predicates and we can use its logic simplification to find minimal distinguishing formulas, whereas Z3 could attempt to find a formula that separates two states.
- Group theory: use **SageMath** (which can use GAP internally) for σ_{sym} to compute orbits when G is non-trivial. Our module might call Sage’s group action functions on the fly (possibly via an API or by running in the same Python process if Sage libraries are imported).
- Causal inference: use existing implementations of PC or FCI, such as **pgmpy** or **DoWhy** in Python, to aid σ_{caus} . We can feed in observational data from X and get a learned causal graph, then do the merging accordingly.
- Trajectory analysis: use libraries for sequence analysis or rough path, e.g. **iisignature** for computing path signatures, or simply numpy/pandas for sequence processing. By building on these, KUSMAN-

Reduce avoids duplicating complex functionality and stays up-to-date with state-of-the-art algorithms. The architecture will provide adapters for these libraries to speak to our System IR. For example, `AlgebraicClosure.apply` might convert K_X into a NetworkX DiGraph, run a community detection or partition algorithm, then translate the result back to our IR (merging states in the partition). This approach ensures efficiency and reliability by utilizing well-tested code.

- **Extensibility:** The design philosophy is *modular and extensible*. New closures can be added by implementing the interface and registering them with a central registry or configuration (imagine a dictionary of name $\rightarrow$ module). Each new closure should specify what invariants it preserves and what structures it requires. For instance, if someone devises a new closure σ_{quantum} for quantum systems, they can integrate it provided it respects the MMK interface and defines how it reduces the base components. The system can then treat it like any other closure. The extensibility also applies to domain-specific adaptations: one can subclass the System IR for particular domains (e.g. an ecology-specific system might include species labels and a specialized closure for food-web trophic equivalence). The framework would still allow combining that with the standard closures. Metadata about closures (like “ σ_i commutes with σ_j ” or “ σ_k requires metric structure”) can be encoded so the pipeline executor knows how to optimize or warn. This modular approach corresponds to KUSMAN-Reduce’s ability to “**register new closure operators with metadata (isomorphism invariants, commutativity, etc.)**” ⁴⁸.

- **User Interfaces:** The library provides both a programmatic API and possibly a high-level language for specifying reductions. In Python, for example, one can simply import the closures and compose them. For instance, using the earlier example:

```
from kusan_reduce import AlgebraicClosure, GeometricClosure, SpectralClosure,
LogicalClosure
X = System.load("model.udif")
reduced = AlgebraicClosure().apply(X)
reduced = GeometricClosure().apply(reduced)
# Or compose them in one call:
reduced2 = (LogicalClosure() << SpectralClosure() << GeometricClosure() <<
AlgebraicClosure())(X)
```

In this pseudocode, `<<` might be an operator to compose closures (making a pipeline that is then called on X). We also provide convenience functions: e.g. `compose([sym, a, g, s, traj], X)` would apply $\sigma_{\text{sym}} \circ \sigma_a \circ \sigma_g \circ \sigma_s \circ \sigma_{\text{traj}}$ ⁴⁴. For typical use cases, domain-specific helpers exist: for instance, `Ecology.reduce(X, closures=[ctx, coh, a, g])` might apply a recommended pipeline for ecological models (contextual, cohomological, algebraic, geometric) ⁴⁴. These higher-level APIs make it easy to invoke the framework without deep knowledge of each closure.

- **Visualization Tools:** While not explicitly listed in the question, an important part of architecture is making results interpretable. We plan to include methods to visualize the **closure lattice** and the reduced models. For example, the library can output a graphviz diagram of the lattice of subsets of $\{a, g, s, \ell, \text{sym}, \text{caus}, \text{ctx}, \text{coh}, \text{traj}\}$, highlighting which closures have been applied in a given reduction and how one can traverse to other combinations. The user might call `X.visualize_lattice()` to

produce a schematic where each node is a partially reduced system and edges are closure applications. The final node (all closures) is $\mathcal{Q}_{\text{ext}}(X)$ at the bottom. This helps in understanding intermediate reductions and the interplay of closures. For the reduced system itself, we can output summary statistics or graphs (like an aggregated state transition graph). We integrate with plotting libraries (matplotlib or Plotly) for showing, say, how σ_s grouped states by spectral coordinates or how σ identified a cycle (perhaps highlighting the cycle in the original graph). All these serve the goal of “visualizing the closure lattice” and the results of reduction, as requested.

Example Usage: To illustrate the architecture in action, suppose a user has a system X representing a stochastic ecological network with spatial layout. The user might want to reduce it by merging indistinguishable species (algebraic closure σ_a), merging species that are in the same connectivity cluster (geometric σ_g), ignoring seasonal context (contextual σ_{ctx}), and preserving topological food-web loops (cohomological σ_{coh}). Using KUSMAN-Reduce, the user can declare this pipeline and execute it in a few lines:

```
from kusan_reduce import closure_alg, compose, system_struct
X = system_struct.load('ecology_graph.udif')           # Load base system
pipeline = [closure_alg.contextual(), closure_alg.cohomological(),
            closure_alg.algebraic(), closure_alg.geometric()]
X_reduced = compose(pipeline, X)
# Apply ctx → coh → a → g in order
X_reduced.save('ecology_reduced.udif')                 # Save the reduced system
```

(This is a simplified pseudocode example; in practice each `closure_alg.X()` returns an object that when called does σ_X). The result `X_reduced` is the quotient system $\mathcal{Q}_S(X)$ for $S=\{\text{ctx}, \text{coh}, a, g\}$. The user can then inspect `X_reduced` to see, for instance, that the number of effective species is much smaller, cycles in the food web are preserved (verified by comparing homology before and after), and context differences have been eliminated. They might also call `X_reduced.describe()` to get a summary of how many states were merged by each closure and check invariant measures (like total biomass preserved, etc.).

Furthermore, if the user wants to compute the **full aggregator** $\mathcal{Q}_{\text{ext}}(X)$ (all nine closures), they could simply do:

```
from kusan_reduce import ALL_CLOSURES
XQ = compose(ALL_CLOSURES, X)
```

where `ALL_CLOSURES` might be a predefined list of all closure modules. This yields the maximally reduced system $\mathcal{Q}_{\text{ext}}(X)$ which is guaranteed to be minimal in information content while preserving each of the nine aspects. The library ensures that $\mathcal{Q}(X) \cdot \mathcal{Q}(X) = \mathcal{Q}(X)$ is idempotent (calling `compose` again will not reduce it further) and that any subset of closures would give an intermediate model lying in the lattice above $\mathcal{Q}_{\text{ext}}$

Finally, the architecture facilitates *analysis* on the reduced models. A user could visualize the closure lattice by retrieving it from the executor: `lattice = compose.get_lattice_diagram(X)`, which returns a

graph structure of reductions. They could then use a plotting utility to draw it. Each node in this lattice has pointers to the corresponding reduced System object (if computed) or can be computed on-demand. This ability to navigate among reduced versions of X underscores the flexibility of KUSMAN-Reduce in exploring trade-offs between different combinations of simplifications.

In summary, KUSMAN-Reduce’s architecture is built to implement the KUSMAN $\mathcal{MMK}$ closure theory in code: it cleanly separates each closure algorithm (making use of optimized libraries where possible), provides a robust execution framework for applying and verifying closures in combination, and offers user-friendly interfaces for specifying reductions and examining the results. The next section describes the **Universal Dimensional Interchange Format (UDIF)**, which complements this architecture by defining how systems and their reductions are serialized and shared.

6.3 UDIF Serialization and Interchange Layer

To support interoperability and long-term reuse of reduced models, we introduce the **Universal Dimensional Interchange Format (UDIF)**. UDIF is a standardized serialization format for $\mathcal{MMK}$ systems, their reductions, and the equivalence relations (quotient maps) produced by closures. It serves as an archival format that captures not only the state of a system but also the history of closures applied and the mapping between original and reduced states. In this section, we formally define the UDIF schema and discuss how it facilitates interchange between different tools and even translation to other representation languages like JSON-LD or RDF. We also provide examples of how a reduced system and its reduction path would be represented in UDIF, and how one can reconstruct $\mathcal{Q}_n(X)$ from such an archive.

UDIF Schema Definition: A UDIF document for a system X is a tuple structure encapsulating all relevant information. Formally, we define:

$$\text{UDIF}(X) := (\text{base_system}, \text{domain_origin}, \text{domain_specific_data}, \text{closure_history}, \text{quotient_map}, \text{meta})$$

Each component is defined as follows:

- **base_system**: A complete description of the original system X in $\mathcal{MMK}$ format. This includes the state set and structure $(|X|, d_X, \mu_X, K_X)$. In UDIF, we encode this in a machine-readable form. For example, **base_system** might contain:
 - A list of state identifiers (each state can be given a unique ID or label).
 - A representation of the metric d_X (could be a matrix of distances or functional description if compact).
 - The measure μ_X (e.g. a list of state weights, or a distribution over states).
 - The transition kernel K_X (e.g. a list of transitions or a sparse matrix). Essentially, **base_system** is the core data needed to reconstruct X as an $\mathcal{MMK}$ object⁴⁹. It must satisfy all $\mathcal{MMK}$ axioms (like metric symmetry/triangle inequality, probabilities summing to 1, etc.)⁵⁰. In a UDIF JSON file, this might appear as a section with keys like `"states": [...], "distance_matrix": [...], "measure": [...], "kernel": [...]`.
- **domain_origin**: An identifier of the *original domain or context* of X ⁵¹. Since $\mathcal{MMK}$ can embed many domain-specific systems (graphs, topological spaces, group systems, quantum

systems, etc.), `domain_origin` specifies what kind of system this was initially. It could be one of a controlled vocabulary, e.g. "Top" for topological space, "Meas" for measure space, "Grp" for group action system, "Hilb" for Hilbert space (quantum) system, "Eco" for an ecological network, "Comp" for a computation/automaton, etc. ⁵¹. This tag helps interpret domain-specific data and also guides what closures are applicable (for example, a pure measure space might not have a meaningful $\text{\textbackslash varsigma_g}$ if no graph structure, whereas a Hilbert space might invite a spectral closure interpretation).

- `domain_specific_data`: A structured record of any additional information specific to the domain of X ⁵². This section's schema varies depending on `domain_origin`. For example:
 - If `domain_origin = "Top"` (topological space), `domain_specific_data` might include a description of the topology $\mathcal{T}_X$ (like a list of open sets or a finite simplicial complex), the fundamental group $\pi_1(X)$ if known, or cohomology ring $H^*(X)$ if precomputed ⁵².
 - If "Grp", it might specify the group G_X acting on X and the action representation ρ_X ⁵² (for instance, a permutation representation).
 - If "Hilb" (quantum system), it could list the Hilbert space dimension or basis H_X , unitary evolution operators U_X or Hamiltonian, and observable sets $\mathcal{O}_X$ ⁵³.
 - If "Eco" (ecological network), it might contain species names, interaction types, etc.
 - If "Comp" (computational model), perhaps the set of logical predicates or automaton structure. Each domain has a predefined schema in UDIF to capture such details, ensuring that if we share a UDIF file, others can parse the domain-specific content correctly. Importantly, `domain_specific_data` does not affect the `base_system` representation (which is domain-agnostic $\mathcal{MMK}$), but it enriches the UDIF with context (so that a user knows, for example, this system originally came from a topology with certain holes, or a quantum system with certain symmetries) ⁴⁹.
- `closure_history`: A record of which closures have been applied to obtain a given reduced system, in chronological or sequential order. This is crucial for reproducibility. We store a list of entries, each entry detailing one closure application. For each closure $\text{\textbackslash varsigma_i}$ applied, we record:
 - The name or index of the closure (e.g. "varsigma_a" or "algebraic").
 - The *timestamp* or step at which it was applied (to allow ordering and possibly timing information).
 - The *invariants preserved* by this closure (a brief summary of what properties remain unchanged). For instance, an entry for $\text{\textbackslash varsigma_g}$ might list "preserved: strong connectivity, distance within components" to indicate geometric structure invariants, or $\text{\textbackslash varsigma_coh}$ might list "preserved: Betti numbers (1:1, 2:0,...)" indicating which Betti numbers or homology features remain ⁵⁴.
 - Possibly a measure of information loss or compression at that step (e.g. reduction in state count or an information metric value, which could be part of `closure_metadata` as described later). Each entry might look like: `{"closure": "varsigma_a", "time": "2025-10-23T01:40:00Z", "invariants": ["Markov probabilities"], "states_before": 100, "states_after": 60}` to denote algebraic closure was applied reducing 100 states to 60 and preserving Markov transition probabilities patterns. By scanning `closure_history`, one can see the *exact sequence* of transformations that led from the original system to the current one. This is more detailed than the

simple list of applicable closures originally in UDIF spec ⁵⁴; it includes ordering and context. If a UDIF file represents $\mathcal{Q}_{\text{ext}}(X)$ (the full reduction), `closure_history` will list all nine closures in some order (which should be immaterial up to commutation), but it documents the path taken. If the UDIF represents a partially reduced system, only those closures used are listed.

- `quotient_map`: Precise *identification data* mapping original states to reduced states. This is essentially the formal record of the equivalence relation. We include this to allow reconstruction of how the quotient was formed. For example, `quotient_map` could be a list or dictionary where each original state ID maps to an ID of the corresponding state in the reduced system. If multiple reduction steps were applied, this map typically reflects the composite effect (original to final). We might also store intermediate maps per closure, but that can be derived by comparing successive entries, so the primary content is original-to-final mapping. E.g., `"quotient_map": {"state1": "classA", "state2": "classA", "state3": "classB", ...}` means in the final reduced system, state1 and state2 ended up merged into classA, state3 is in classB, etc. The IDs "classA", "classB" would correspond to new state labels in the reduced system (which are listed in `base_system` of the reduced model if we include it). In UDIF, since we also want a self-contained representation of the *reduced system*, we often embed the reduced system's definition as well. There are two ways to handle this:
 - The UDIF document can describe **one particular stage** (for instance, the final reduced system). In that case, `base_system` would actually describe the reduced system's components, *not* the original. But then we need somewhere the original reference. In that scenario, `quotient_map` would map original IDs to reduced IDs, giving the link back. We may also store original base system separately or have an external reference.
 - Alternatively, UDIF can be used to describe the entire transformation from original to final. In this approach, `base_system` is the original, and we include an additional field like `reduced_system` for the final outcome. The schema given in earlier references ⁵⁵ did not explicitly show `reduced_system`, possibly because UDIF might be used to serialize any object in the category (in that usage, one UDIF corresponds to one system). To accommodate a full reduction trace, we might extend the schema. For clarity here, we describe UDIF as capturing one system and its provenance. So if we are saving the *result* of reductions, `base_system` will refer to that result. The `quotient_map` then acts as a bridge to the original (which might be referenced via a separate UDIF document or an ID if known). In practice, a UDIF package could allow bundling the original and reduced together. For this section, we assume the UDIF file in question is describing a reduced system and contains enough info to link back to the original via `quotient_map`. This mapping is critical for tasks like **reconstructing $\mathcal{Q}_n(X)$** : if we have an archived reduction, we can take a new copy of the original X , apply the `quotient_map` to collapse it, and verify we get the same reduced structure. It also allows transferring results (e.g., if one wants to lift a simulation result from the reduced model back to original state labels, the map is needed).
- `metadata`: Additional metadata section which can include **provenance and annotations**. Provenance information might cover:
 - The author or software that performed the reduction (e.g. `"generated_by": "KUSMAN-Reduce v1.0"`).
 - Timestamps of creation and modification.

- Descriptive tags or natural-language descriptions of the system or the purpose of reduction (e.g. `"description": "Reduced model of XYZ preserving homology and spectral properties"`).
- **Closure metadata** summarizing efficiency or information content ⁵⁴. For instance, an information-theoretic metric like entropy or mutual information of the system state could be recorded before and after reduction, to quantify information loss. We might list invariant measures (like "total mass preserved = 100") or semantic guarantees (like "logical predicates remaining: ..."). This `metadata` can thus include what was referred to as "efficiency, semantic invariants, information metrics" in earlier descriptions ⁵⁴. Essentially, anything that doesn't fit into the structural fields above can go here. For example, if varsigma_s was applied, we might store the numerical values of the top k eigenvalues retained vs dropped as part of metadata, to indicate fidelity. If $\text{varsigma}_{\text{coh}}$ was applied, we might store the barcode of persistent homology so that someone reading the UDIF knows which topological features were preserved exactly. Additionally, `metadata` may contain links or references: for example, a DOI or URN for the original dataset, or references to academic articles. It could also contain `domain_origin`-specific info that doesn't go in the structured `domain_specific_data`, such as naming of states or interpretative labels (like mapping state IDs to actual species names in an ecological model). Contextual annotations might also include notes like "this reduction was performed under assumption X" or "some states were removed entirely due to no influence" if that happened.

In summary, UDIF's schema captures: the core system (base data), the domain context, the full history of closure operations applied, the mapping from original to reduced, and extra metadata. A simpler view of this (excluding history) was given by the tuple in previous work ⁵⁵, but here we explicitly incorporate the *history and mapping*, which are key for interchange in the context of reductions.

Well-Formedness and Semantics: A UDIF file must be **self-consistent** and valid. Formally:

- The `base_system` description must constitute a valid MMK object on its own (satisfying metric, measure, kernel properties) ⁵⁰. Tools reading UDIF can reconstruct a system Y from `base_system` and ensure it lies in MMK .
- The `closure_history` listed should, when applied in sequence to the *original* X , produce a system isomorphic to the `base_system` described. If `base_system` is the reduced system, applying the closures to original should yield that same result (this consistency one can verify if original X is available, or one can rely on the stored `quotient_map` as a certificate).
- The `quotient_map` should be a surjection from original states to reduced states and its kernel (the set of pairs mapping to same reduced state) should correspond exactly to the equivalence relation induced by composing the closures in `closure_history`. In other words, if x_1 and x_2 map to the same reduced state, then for each closure in history, x_1 and x_2 were not distinguished by that closure (they ended in the same intermediate class at each step). This typically holds by construction if the map is derived during reduction.
- Any invariants or metadata claims should be consistent: e.g. if metadata says "homology class count preserved: 2 loops", one should indeed find 2 loops in the reduced system if they were in original.
- As part of UDIF semantics, we expect that **serializing then deserializing** returns an equivalent object (identity up to isomorphism) ⁵⁶. Likewise, if one serializes an original X to UDIF and then reduces by reading `closure_history`, one should get the same reduced UDIF as given (this relates to the functorial properties below).

There is an implicit **category** at play here: let $\mathbf{UDIF}$ be the category of UDIF-encoded objects (where morphisms could be trivial, or perhaps translations as described next). There is a natural serialization functor $S: \mathcal{MMK} \rightarrow \mathbf{UDIF}$ that takes a mathematical object and produces its UDIF data, and a deserialization functor $D: \mathbf{UDIF} \rightarrow \mathcal{MMK}$ that reconstructs the object. We require $D \circ S = \mathrm{Id}_{\mathcal{MMK}}$ (lossless serialization) and $S \circ D = \mathrm{Id}_{\mathbf{UDIF}}$ (no data lost or changed by a round-trip) ⁵⁶. In practice, $D \circ S(X)$ yields exactly X , and $S \circ D(\text{UDIF})$ reproduces the same UDIF content ⁵⁶. This means UDIF is a faithful representation of $\mathcal{MMK}$.

Translation to Other Formats: One of UDIF's goals is to enable **interchange** not just within KUSMAN tools but across domains and platforms. To that end, we consider UDIF as a kind of universal data model that can be translated into more widely used formats:

- **JSON-LD:** UDIF can be seen as a JSON document (indeed our examples imply a JSON structure). JSON-LD (JSON for Linked Data) is essentially JSON with a context that links keys to IRIs (Internationalized Resource Identifiers) for semantic meaning. We can define a JSON-LD context for UDIF that maps terms like "base_system", "states", "kernel", "closure_history", etc., to URIs in an ontology (perhaps an ontology we create for KUSMAN). By doing so, a UDIF file can be interpreted as part of the semantic web. For example, we might have a context where "base_system": "http://kusman.org/udif#base_system", "varsigma_a": "http://kusman.org/closure#algebraic" and so forth. With JSON-LD, one can query UDIF data using SPARQL or integrate it with other linked data (for instance, linking a state ID to an entity in another knowledge base like a species name in an ontology, via the domain-specific data). The translation functor $\tau_{\text{UDIF} \rightarrow \text{JSON-LD}}$ would thus take the UDIF JSON structure and augment it with a context and possibly transform certain structures into graph-friendly form (e.g. the `quotient_map` could be turned into a set of RDF triples like `<originalState> <mappedTo> <reducedState>`). The result is that the UDIF content becomes part of an RDF graph when interpreted via JSON-LD.

- **Pure RDF/Turtle:** Alternatively, one can create an RDF representation directly. The idea is to have classes like `MMKSystem`, `State`, `ReducedState`, `ClosureApplication` etc., as RDF classes, and properties like `hasState`, `hasTransition`, `mergedInto` (for quotient map relations), etc. For each element of UDIF, we'd create triples. For example:

- Each state $x \in |X|$ becomes a resource (e.g. `X#state1`) of type `State`.
- The metric might be represented by triples like `(state1, hasDistanceTo, state2) = d_X(state1, state2)` if discrete, but that's unwieldy; more likely, we would drop the full matrix and just store the fact the metric exists, leaving actual values in data structure.
- Transition probabilities could be triples like `state1 transitionProb( state2, 0.5 )` but RDF doesn't natively support tuples with probabilities, so we might create a node for each transition or use reification to attach probability value.
- The closures applied can be captured as a sequence of blank nodes or resources: e.g. `ReductionStep1 a ClosureApplication; closureType "varsigma_a"; preserves "behavior"; reducedStateCount 60` etc., and link them in order.
- The mapping from original to reduced can be a property, e.g. `originalState mappedTo reducedState`. This RDF mapping can be complex, but it allows using the rich tooling of semantic

web to query and reason about the reduction. For example, one could query: "give me all original states that ended up in a reduced state which has property P" or "which closures were applied after a symmetry closure?" etc.

- **Category-Theoretic Formats:** By this we mean possibly using categorical data exchange languages or formats that capture the mathematical structure. One idea is to use a format like **COMPOSE** (if it existed for category theory) or simply to describe the system as an object in a category in a human-readable but structured way (maybe something like a variant of *.cat files or something*). *Another interpretation is to leverage existing category theory tooling: for example, represent the $\mathcal{MMK}$ object and its quotient as an arrow in some category (the inclusion $X \hookrightarrow \mathcal{MMK}$ is a morphism in $\mathcal{MMK}$). We could imagine a functor that takes UDIF data and constructs an object in, say, a Category Theory Markup Language (if there is one). However, since that is quite abstract, a more concrete approach is using tools like the Functorial Data Migration* approach in category theory databases. One could design a schema (category) that has entities for states, transitions, etc., and then UDIF data populates an instance (set-valued functor). Then translating to another category (like one for a different domain) is an instance of a functor (which might correspond to the domain translation functors mentioned in the UDIF formal paper ⁵⁷ ⁵⁸ but here target could be something like a category of semantic triples or similar). This is quite advanced, so practically, JSON-LD and RDF cover most needs by expressing UDIF in graph form.*

We can formalize the notion of translation functors as: for any target representation language L that can express the components of UDIF, define a mapping $\tau_{\{\text{UDIF}\} \rightarrow L}: \mathbf{UDIF} \rightarrow L$. If L is itself viewed as a category of data (like RDF graphs as objects and certain mappings as morphisms), then τ is a functor. For example, $\tau_{\{\mathbf{UDIF}\} \rightarrow \mathbf{RDF}}$ would map a UDIF document U to an RDF graph G . We aim for these translations to be **information-preserving** or at least faithful. In the case of JSON-LD and RDF, since they are more general, we should be able to embed UDIF fully (maybe with some verbosity).

One important result from earlier work is that translation functors commute with closure operations ⁵⁸. In our context, this means if you translate a UDIF representing X to, say, RDF, and then in that RDF apply an analogous notion of closure (if defined), it is the same as first applying the closure to X (getting $\mathcal{MMK}(X)$ and its UDIF) and then translating. However, practically we won't be applying closures in RDF space; rather, this theoretical guarantee implies that our translations do not break the structure: the fact that x_1 and x_2 were merged by $\mathcal{MMK}_i$ is representable and preserved in the RDF. The commutative diagram from the UDIF formalization ⁵⁸ conceptually ensures no information about closures is lost in translation: the `closure_history` and `quotient_map` in UDIF can be fully represented in JSON-LD or RDF such that anyone analyzing those formats can infer the same quotient.

Example UDIF Instances: Let us illustrate how a reduced system and its derivation might appear in UDIF. Consider a simple system X with 5 states $\{1,2,3,4,5\}$ that we reduce by first applying $\mathcal{MMK}_a$ (algebraic closure) and then $\mathcal{MMK}_g$ (geometric closure). Suppose $\mathcal{MMK}_a$ merges $\{1,2\}$ (they were behaviorally identical) and merges $\{4,5\}$, leaving $\{3\}$ alone. So after $\mathcal{MMK}_a$, we have classes $A=[1,2]$, $B=[3]$, $C=[4,5]$. Then $\mathcal{MMK}_g$ on that quotient perhaps finds that A and C are strongly connected (just as an example) so it merges them into one class AC , while B remains separate. The final quotient $\mathcal{MMK}_g(\mathcal{MMK}_a(X))$ thus has two states: AC (representing $\{1,2,4,5\}$) and B (representing $\{3\}$). Now the UDIF for the final reduced system might look like (in pseudo-JSON with comments):

```

{
  "base_system": {
    "states": ["AC", "B"],
    "distance_matrix": [[0, d_AC_B], [d_AC_B,
0]], // some distance between AC and B
    "measure": {"AC": 0.8, "B":
0.2}, // aggregated from original (say states1,2,4,5 had total
0.8 prob, state3 0.2)
    "kernel": {
      "AC": {"AC": 0.9, "B": 0.1},
      "B": {"AC": 0.0, "B": 1.0}
    }
  },
  "domain_origin": "Meas",
  "domain_specific_data": {
    // Perhaps original had a sigma-algebra or something; if none, this could be
empty or minimal
    "sigma": "..."
  },
  "closure_history": [
    {"closure": "varsigma_a", "time": "2025-10-23T01:40:00Z",
      "invariants": ["transition probabilities"], "states_before": 5,
"states_after": 3,
      "merged_blocks": [["1","2"], ["4","5"]] },
    {"closure": "varsigma_g", "time": "2025-10-23T01:40:01Z",
      "invariants": ["connectivity"], "states_before": 3, "states_after": 2,
      "merged_blocks": [["A","C"]] }
  ],
  "quotient_map": {
    "1": "AC",
    "2": "AC",
    "3": "B",
    "4": "AC",
    "5": "AC"
  },
  "metadata": {
    "original_id": "X001",
    "generated_by": "KUSMAN-Reduce 1.0",
    "description": "Reduced by algebraic then geometric closure.",
    "info_loss": "KL_divergence: 0.05", // e.g., some measure of information
lost
    "semantic_checks": ["Idempotence verified", "No logical predicates in
input"]
  }
}

```

This example shows the major elements. The `closure_history` explicitly lists what was merged at each step (here we added a `merged_blocks` field for clarity, though in formal UDIF we might not include it or we might include it as part of metadata per step). The `quotient_map` clearly shows which original state ended where. A consumer of this UDIF could, for instance, look at `closure_history[0]` and know it was σ_a and see it produced 3 intermediate states, then `closure_history[1]` applied σ_g . If needed, they could reconstruct the intermediate 3-state system by taking original 5 states and grouping according to the first step's `merged_blocks`: that intermediate system could be described similarly. But since we've given the final, one can verify that if we start from original 5-state XX (which presumably could be retrieved via `original_id: X001` linking to a separate UDIF or dataset), and we apply an algebraic closure algorithm (we know what it means, or we could read invariants and guess it was bisimulation) we'd get states A,B,C, and then apply geometric closure to get AC and B. The `quotient_map` directly tells us final grouping. Thus, everything needed to trust and reuse this reduction is present.

Reconstructing $\mathcal{Q}_n(X)$ from UDIF: If a UDIF file encodes a fully reduced system with closure history, how do we recover $\mathcal{Q}_n(X)$? There are two interpretations: (1) $\mathcal{Q}_n(X)$ is the reduced system itself, which is already in `base_system` — in that sense, UDIF already *is* a representation of $\mathcal{Q}_n(X)$. But (2) we might want to verify it by applying closures to the original XX ourselves. Using UDIF, one can do the following:

- Retrieve or reconstruct the original system XX . This could be done if the UDIF either contains the original in `domain_specific_data` or if we have a reference to original (like `original_id: X001`). Suppose we have XX at hand (maybe from another UDIF or input file).
- Read the `closure_history` sequence. For each entry, invoke the corresponding closure operator on your local copy of XX , in that order. Because the closures commute, the order shouldn't matter, but following the recorded sequence will reproduce exactly the archived path (which might be important if approximations were used). After all steps, you should obtain a system X_{final} .
- Compare X_{final} with the `base_system` in the UDIF. They should match exactly (or be isomorphic). The UDIF essentially claims that they will match ⁵⁶. If you get a different result, either the UDIF was incomplete or the implementations differ. But ideally, with KUSMAN-Reduce using the same closure definitions, you get the same quotient.
- Alternatively, a simpler reconstruction is to use `quotient_map` directly: take the original XX and perform a manual quotienting by the equivalence relation given by that map. That is, create a new system whose states are the unique values on the right side of `quotient_map` (the final labels), and define its metric, measure, kernel by aggregating original states according to those classes (essentially exactly how one constructs a quotient object mathematically). This bypasses re-running the closures sequentially and directly uses the end result of the equivalence relation. It is a one-step reconstruction of $\mathcal{Q}_n(X)$. In many cases, this is more robust because it doesn't depend on having the same algorithm implementations; it uses the explicit mapping. The UDIF provides everything needed for that: the mapping and the original XX . Indeed, one could treat quotient_map as a morphism in $\mathcal{MMK}$ from XX onto the reduced system, and then constructing the coequalizer quotient is a straightforward operation given the map.

Thus, UDIF ensures that knowledge of the reduced system is not lost and is reproducible. If multiple parties reduce the same system independently, they can compare UDIF files: if the closure history and quotient map are the same, they reached the same $\mathcal{Q}_n(X)$, even if via different paths (commutativity guarantees uniqueness of the true $\mathcal{Q}_n(X)$ up to isomorphism ⁴¹ ⁵⁹). If histories differ or

approximations differ, UDIF will make those differences evident (different merges or a different final mapping).

Interoperability Use-Case: Suppose a researcher in biology reduces a complex ecological model with KUSMAN-Reduce and shares the UDIF. A researcher in mathematics could load this UDIF into their category-theoretic environment, confirm the homology was preserved (looking at metadata or recomputing it from `base_system`), and maybe translate the UDIF to a format for a theorem prover to verify some property. Or a computer scientist could convert the UDIF to a JSON-LD, merge it with an RDF dataset of species traits, and query whether species merged in the quotient also share certain traits (testing if the reduction aligns with external knowledge). The *universal* nature of UDIF means it's not tied to a single programming language or tool; it's a **lingua franca** for reduced models.

In practice, we implement UDIF support in KUSMAN-Reduce such that every reduction pipeline can output a UDIF document. Parsers and serializers use JSON schema definitions to validate the structure ⁶⁰, and provide optional compression (Protocol Buffers or HDF5) for large numeric parts. The **parsing/translation modules** can then provide export functions: e.g. `udif.to_jsonld()` to get JSON-LD text, or `udif.to_rdf()` to get an RDF/Turtle string ⁶⁰. We ensure that closures and UDIF interplay seamlessly, so that applying a closure in software updates the UDIF representation appropriately, and conversely, reading a UDIF with a closure history can trigger the pipeline executor to re-run those steps for verification ⁶¹.

In conclusion, the UDIF interchange layer provides a rigorous yet practical way to serialize MMK systems and their multifaceted reductions. It formalizes the **syntax** of stored data (so any tool can parse it) and the **semantics** (so the mathematical meaning is clear and preserved) ⁵⁶. By including closure application history, invariants, and quotient mappings, it not only records the end result but the reasoning path, enabling trust and reproducibility. Through translation functors to formats like JSON-LD and RDF, UDIF ensures that the rich information in reduced models can be integrated into broader knowledge ecosystems, fulfilling the promise of truly universal model reduction that is *shareable* and *verifiable* across disciplines.

1	9	12	13	14	15	17	18	21	22	24	25	26	27	28	29	30	32	33	34	35	36	37	38	39	40	41	47	59
b502dccb.md																												
file:///file-ETyTX35yWkEM1QPvXuTajb																												
2	3	4	5	6	7	8	10	11	16	19	20	23	31	42	43	44	45	46	48	0dae3884.md								
file:///file-5JRgRpSHUSG1AVExsAVSE4																												
49	50	51	52	53	54	55	56	57	58	60	61	47ef8582.md																
file:///file-2VGuo7JcwXkBWGmUrDH4T4																												